Java Spring · Week 4

A running server, a layered API

Spring Boot, HTTP, REST and the three layers

Eng. Mohamed Shehab

Faculty of Computers and Information, Menoufia University

By the end of this session

Boot

A Spring Boot project from start.spring.io, and what is inside it

Layers

Three layers first: repository, service, then the controller

HTTP

What a request carries, and the server that answers it

REST

GET, POST and DELETE on one resource, each with the right status

First, the misconception

Boot is Spring, already set up

Spring Framework

The container from Week 3, plus web, data, transactions. Powerful and unopinionated.

Spring Boot

The same framework with defaults, a server included and a starter for each job.

You do not write a Spring Boot pom by hand

```
start.spring.io

Project Maven Language Java
Boot 4.1.2 Packaging Jar
Group com.eventia Artifact eventia
Java 21

Dependencies Spring Web
```

In IntelliJ Ultimate or STS

New Project has Spring Initializr built in

In IntelliJ Community

It does not. Use the website and open the unzipped folder

eventia / pom.xml, as start.spring.io wrote it

Where did the version numbers go?

```
<parent>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.2</version> ← the only version in the file
</parent>

<dependency>
  <artifactId>spring-boot-starter-webmvc</artifactId> ← no version
</dependency>
```

The parent holds a tested set of versions. You name the library, it names the number.

eventia / pom.xml

One line brings thirty eight JARs

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.2</version>
</parent>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
```

Java Spring · Week 4 · 06

Counted with dependency:tree on this project. Tomcat, Jackson, logging and the web stack, all versioned by the parent.

eventia / EventiaApplication.java

Ctrl+Click on the annotation

```
@SpringBootApplication
public class EventiaApplication {
    public static void main(String[] args) {
        SpringApplication.run(EventiaApplication.class, args);
    }
}
```

Inside it	You already know it from
@Configuration	AppConfig in Week 3
@ComponentScan	the scanner you wrote in Week 2
@EnableAutoConfiguration	nothing yet. This is the only new thing

Settings live in a file, not in code

application.properties

```
spring.application.name=eventia

server.port=8080
```

The same keys as application.yml

```
spring:
  application:
    name: eventia

server:
  port: 8080
```

One key per line. YAML says the same thing nested. Pick one format per project.

Before any endpoint: where each job lives

Three layers, one reason to change each

web

Controller

Speaks HTTP. Reads the request, picks a status, returns a body.

service

Service

Holds the rules. Knows nothing about HTTP at all.

repository

Repository

Talks to storage. Today a list, later a database.

When the storage becomes a real database, only the bottom box changes.

eventia / repository/InMemoryEventRepository.java

The bottom layer: a list, for now

```
@Repository
public class InMemoryEventRepository {

    public List<EventDto> findAll() { ... }
    public Optional<EventDto> findById(Long id) { ... }
    public EventDto save(String title, int capacity, LocalDateTime startsAt) { ... }
    public boolean deleteById(Long id) { ... }
}
```

The methods are already named the way a database repository is named. That is deliberate.

eventia / service/EventService.java

The rule: no HTTP words in here

```
@Service
public class EventService {

    private final InMemoryEventRepository repository;

    public List<EventDto> findAll() {
        return repository.findAll();
    }

    public Optional<EventDto> findById(Long id) {
        return repository.findById(id);
    }
}
```


No ResponseEntity, no status code, no HttpServletRequest. It returns what it found, and the controller decides what that means over

Before any Java: what actually travels

A request and a response are just text

```
GET /api/events/1 HTTP/1.1 ← method, path
Host: localhost:8080
Accept: application/json ← headers

(a GET has no body)
```

```
HTTP/1.1 200 ← status
Content-Type: application/json

{"id":1,"title":"Spring Boot Workshop"} ← body
```

Four parts going in: method, path, headers, body. Three coming back: status, headers, body.

The first word of the request

The method says what you want done

Method	Means	Body?	Success
GET	Give me this. Change nothing	no	200 with the data
POST	Make a new one from this body	yes	201 and where it is
PUT	Replace this one with this body	yes	200 or 204
DELETE	Remove this one	no	204, nothing to say

GET can be repeated a thousand times and nothing changes. POST twice makes two.

The ones you will actually use

Status codes worth knowing

Code	Name	When you send it
200	OK	A successful GET, or an update that returns the new state
201	Created	A POST that made something. Add a Location header
204	No Content	A successful DELETE, or an update with nothing to return
400	Bad Request	The body or a parameter is wrong. The client can fix it
404	Not Found	No such resource
409	Conflict	The request is valid but the state says no. Used once booking has rules
500	Server Error	You broke. Never send details with it

Before any of this existed

One request, one operating system process

CGI, 1993

The server forks a program per request. It starts, prints HTTP to stdout, and dies.

Servlet, 1997

One object, loaded once, handling every request on a thread pool.

Servlets exist because starting a process per request does not scale. Everything after that is refinement.

What replaced the program-per-request

A servlet is a Java class that answers requests

You write

A class that extends HttpServlet

Override doGet for GET, doPost for POST. Nothing else is required.

It receives

HttpServletRequest

Everything the client sent: method, path, parameters, headers, body.

It fills

HttpServletResponse

Everything that goes back: the status, the headers, the body.

Tomcat makes one object of it, keeps it alive, and calls it for every request on its path.

Every line of the HTTP message has a method

Two objects: what came in, what goes out

HttpServletRequest	gives you	HttpServletResponse	sets
`getMethod()`	"GET"	`setStatus(200)`	the number on the first line
`getRequestURI()`	"/hello"	`setContentType("text/plain")`	the Content-Type header
`getParameter("name")`	"Ahmed" from ?name=Ahmed	`setHeader("Location", ...)`	any other header
`getHeader("Accept")`	one header	`getWriter().write(...)`	the body
`getReader()`	the body, as text		

Left is the request slide read by Java. Right is the response slide written by Java.

eventia / web/HelloServlet.java

A servlet, written by hand, in our project

```
public class HelloServlet extends HttpServlet {

    public void init() {
        System.out.println("HelloServlet init, this runs once");
    }

    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws IOException {
        System.out.println("doGet on thread "
                + Thread.currentThread().getName());
        String name = request.getParameter("name");
        response.setContentType("text/plain");
        if (name == null) {
            response.setStatus(400);
            response.getWriter().write("name is required");
            return;
        }
        response.setStatus(200);
        response.getWriter().write("Hello, " + name);
    }
}
```

Registered in EventiaApplication

```
@Bean
public ServletRegistrationBean<HelloServlet>
        helloServlet() {
    return new ServletRegistrationBean<>(
            new HelloServlet(), "/hello");
}
```

Sent with curl

```
$ curl .../hello?name=Ahmed
Hello, Ahmed 200

$ curl .../hello
name is required 400

$ curl -X POST .../hello
405 Method Not Allowed
```

Printed in the console

```
HelloServlet init, this runs once
doGet on thread http-nio-8080-exec-1
doGet on thread http-nio-8080-exec-2
```

The interface everything on this slide deck sits on

A servlet is three methods

```
init(ServletConfig) once, at startup
service(request, response) every request, on a pooled thread
    → doGet / doPost / doPut / doDelete
destroy() once, at shutdown
```

Tomcat calls these for you: init once, service on every request from its thread pool, destroy at shutdown.

Who keeps that one object alive, and calls it?

Tomcat: the program that runs your servlets

1

Listens

Keeps port 8080 open and waits.

2

Reads

Turns the HTTP text into a request object.

3

Hands off

Takes a thread from its pool of 200.

4

Calls

Finds the servlet for the path and calls service().

5

Writes

Turns the response object back into HTTP text.

A web server plus a servlet container. Jetty and Undertow do the same job. Your code only ever sees step 4.

Stop one

There is a web server inside your JAR

```
Tomcat started on port 8080 (http) with context path '/'
Started EventiaApplication in 1.5 seconds
```

You installed nothing. Tomcat arrived with spring-boot-starter-webmvc, and Boot started it because the class was on the classpath.

Where the server lives, and who starts it

WAR into a server, or a server inside a JAR

	The old way: WAR	What we do: executable JAR
You ship	A WAR file, classes and web resources	A JAR with your code and Tomcat inside
Tomcat is	Installed on the server, shared	A dependency, versioned in your pom
Started by	The operations team, separately	java -jar app.jar
Upgrading it	A server-wide change, every app at once	One line in your pom

Six stops between the browser and your code

1

Tomcat

Listens on the port, reads the bytes.

2

DispatcherServlet

The single front door for every request.

3

HandlerMapping

Which method matches this URL and verb?

4

Your controller

The only part you write.

5

MessageConverter

Turns the returned object into JSON.

6

Tomcat again

Writes status, headers and body back.

You write one of these six. Boot gives you the other five.

Stop two

One front door, not one per URL

Without a front controller

Every URL is its own servlet, and every cross-cutting concern is copied into all of them.

With DispatcherServlet

One servlet takes everything, then routes. Logging, security and error handling live there.

Everything that has to happen on every request happens once, in one place.

Stop three

How the URL finds your method

Method and path	Goes to	Because of
GET /api/events	findAll()	@GetMapping with no path
GET /api/events/1	findById(1)	@GetMapping("/{id}")
POST /api/events	create(body)	@PostMapping
DELETE /api/events/1	delete(1)	@DeleteMapping("/{id}")

Built once at startup by reading your annotations, the same reflection you wrote in Week 2.

eventia / web/EventController.java

Stop four: the part you write

```
@Controller
@RequestMapping("/api/events")
public class EventController {

    private final EventService service;

    public EventController(EventService service) {
        this.service = service;
    }
}
```


@Controller tells Spring: this class answers web requests. Same family as @Service and @Repository.

eventia / web/EventController.java, still a plain @Controller

The first endpoint, and a surprise

```
@GetMapping
public List<EventDto> findAll() {
    return service.findAll();
}
```

```
$ curl http://localhost:8080/api/events
{"status":404,"error":"Not Found","path":"/api/events"}
```

The method exists and the path matches. So why 404?

eventia / web/EventController.java

@ResponseBody: what you return is the body

```
@GetMapping
@ResponseBody
public List<EventDto> findAll() {
    return service.findAll();
}
```

```
$ curl http://localhost:8080/api/events
[{"id":1,"title":"Spring Boot Workshop","capacity":40,
  "startsAt":"2026-11-05T08:15:11.0244691"},
 {"id":2,"title":"Docker for Beginners","capacity":25, ...}] 200
```


No @ResponseBody: the return value is the name of a page. With it: the return value is the data, written as JSON.

Ctrl+Click on @RestController

@RestController = @Controller + @ResponseBody

Before, w4-10

```
@Controller
@RequestMapping("/api/events")
public class EventController {

    @GetMapping
    @ResponseBody
    public List<EventDto> findAll()
    ...
}
```

After, w4-11

```
@RestController
@RequestMapping("/api/events")
public class EventController {

    @GetMapping
    public List<EventDto> findAll()
    ...
}
```

Inside it

```
@Controller
@ResponseBody
public @interface RestController
```

	@Controller	@RestController
The method returns	the name of a page	the data itself
Spring sends back	an HTML page	JSON
Used for	websites with server-side pages	APIs, like ours

eventia / web/EventController.java

Reading a value out of the URL

```
@GetMapping("/{id}")
public ResponseEntity<EventDto> findById(@PathVariable Long id) {
    Optional<EventDto> event = service.findById(id);
    if (event.isPresent()) {
        return ResponseEntity.ok(event.get());
    }
    return ResponseEntity.notFound().build();
}
```

The name in the braces and the parameter name must match, or you tell Spring the name yourself.

The decision inside that method

404, not 200 with nothing in it

200 with an empty body

The client has to guess. Some will treat it as success.

404 Not Found

Unambiguous. Every HTTP client already knows what it means.

```
$ curl -o /dev/null -w "%{http_code}" http://localhost:8080/api/events/99
404
```

eventia / web/EventController.java

Taking a body in, sending a location back

```
@PostMapping
public ResponseEntity<EventDto> create(@RequestBody CreateEventRequest request) {
    EventDto created = service.create(request);
    return ResponseEntity
                .created(URI.create("/api/events/" + created.id()))
                .body(created);
}
```

```
HTTP/1.1 201
Location: /api/events/3
Content-Type: application/json

{"id":3,"title":"Intro to Redis","capacity":30,"startsAt":"2026-11-01T18:00:00"}
```

eventia / web/EventController.java

A response with no body at all

```
@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void delete(@PathVariable Long id) {
    service.delete(id);
}
```

```
$ curl -o /dev/null -w "%{http_code}" -X DELETE http://localhost:8080/api/events/2
204
```

HelloServlet next to EventController

What you did by hand, Spring now does

The job	In HelloServlet, by hand	In the controller
Map the URL	`new ServletRegistrationBean<>(..., "/hello")`, one path per servlet	`@GetMapping("/{id}")` on each method
Read the input	`request.getParameter("name")`	`@PathVariable`, `@RequestBody`
Choose the status	`response.setStatus(400)`	`ResponseEntity`, `@ResponseStatus`
Set the content type	`setContentType("text/plain")`	the MessageConverter, from what you return
Write the body	`getWriter().write(...)`, a string you build	Jackson turns the object into JSON
Refuse other methods	405 from HttpServlet	405 from the DispatcherServlet

The controller still runs inside a servlet: the DispatcherServlet. Spring writes that plumbing once.

Stop five

Who turned the object into JSON?

Object to JSON

Going out

The returned object is handed to a MessageConverter, which writes JSON into the response.

JSON to object

Coming in

@RequestBody sends the request body through the same converter, the other way.

Jackson, the library from Week 1, now version 3. It came with the starter this time.

Two classes that look the same today

Why the API shape is its own class

	The DTO	The entity, once a database arrives
Owned by	The API contract	The database schema
Changes when	Clients need a different shape	The tables change
Contains	Only what the client should see	Everything a row has, including what it should not
Breaking it	Breaks every client	Breaks a migration

Be honest about what this is

Restart the application and the data is gone

```
private final List<EventDto> events = new ArrayList<>();
```

A list in a field is not storage. It is a placeholder that lets us finish HTTP today.

Six things to take away

1

Boot is Spring with defaults, a bundled server and a starter per job.

2

A request is a method, a path, headers and a body. A response is a status, headers and a body.

3

The method says what you want. The status says what happened.

4

One servlet answers every request, so anything in a field is shared.

5

6

Next week

Validation and errors: an API that refuses bad input, politely

Eng. Mohamed Shehab

Faculty of Computers and Information, Menoufia University

mohamed.shehab.fci@gmail.com